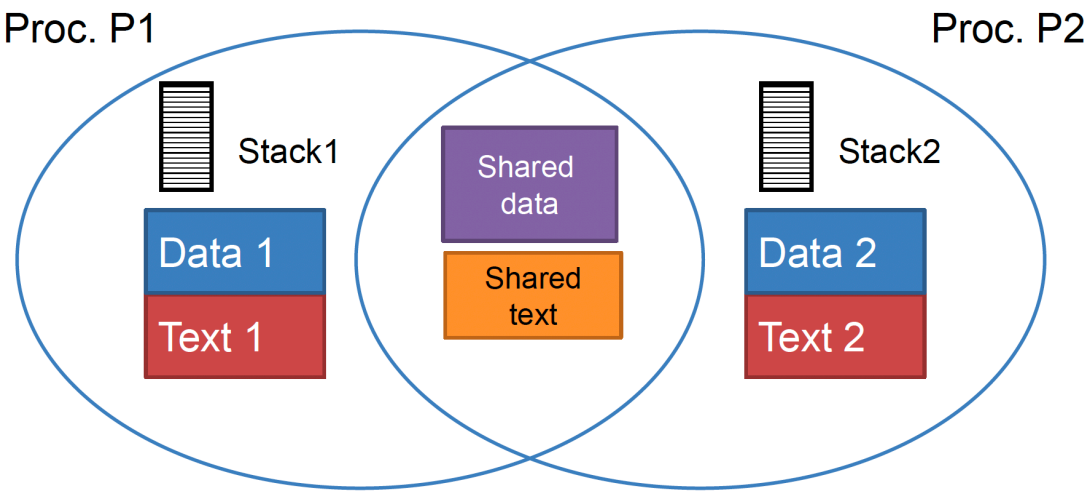
Synchronization

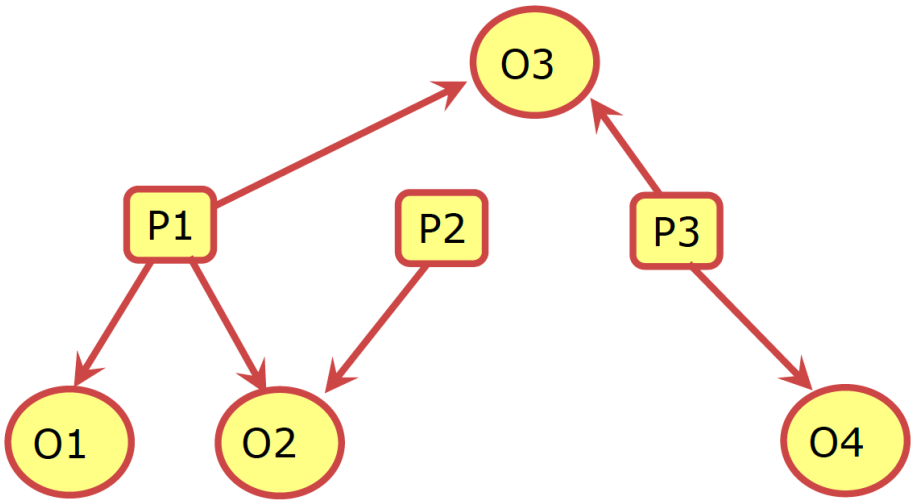
Description

Global vs Local model

Global environment

- Processes/threads can share data
- Cooperation via ***shared memory variables***



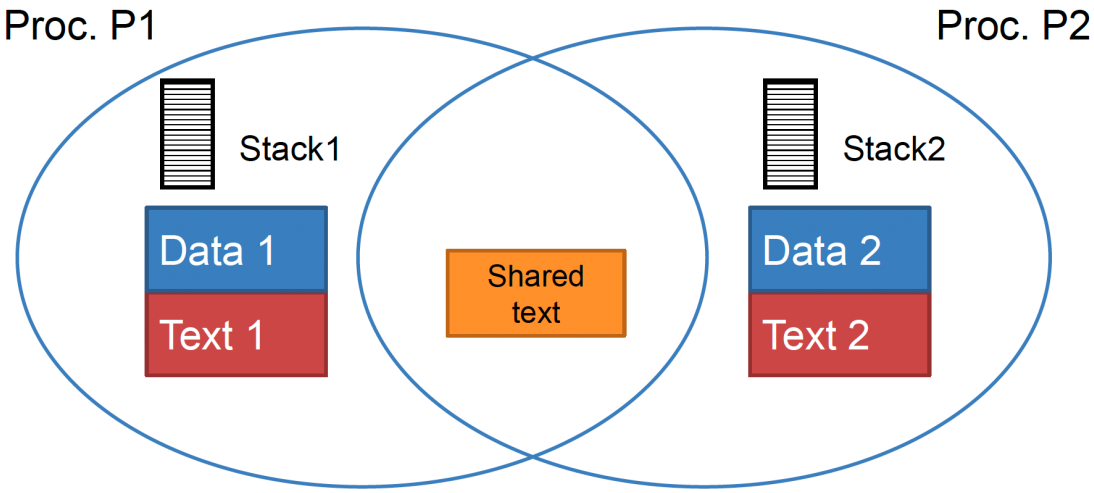


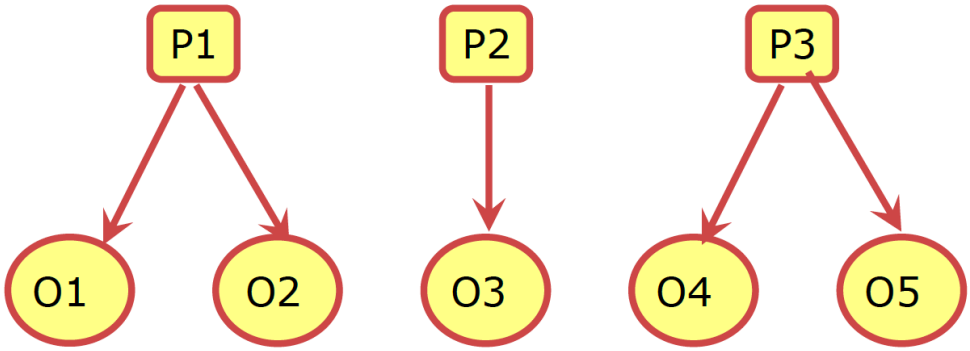
O1, O4 private objects
O2, O3 shared objects

- competition, cooperation

Local environment

- processes/threads do not share data
- No shared memory
- Cooperation via ***explicit messages***

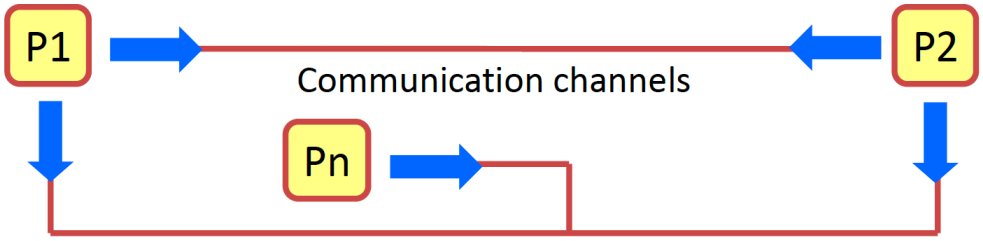




O1-O5 are private objects

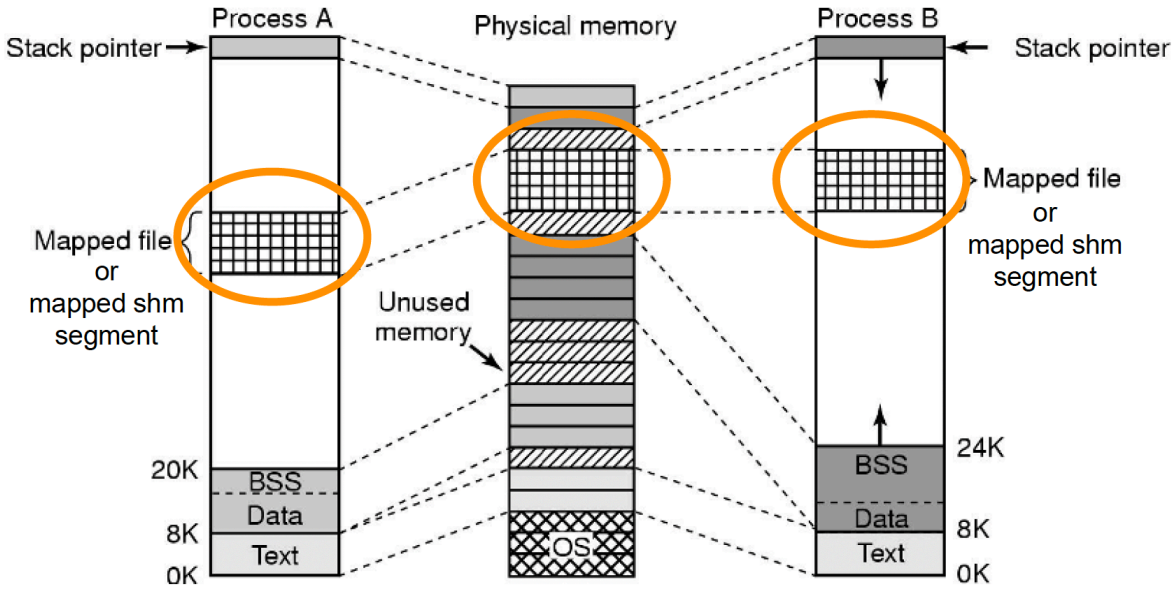
Competition through server processes

Cooperation through explicit communication



Cooperation (communication, synchronization) by means of message passing

Segment sharing using memory mapping in Unix



Synchronization Motivation

When multiple threads concurrently read or write shared memory variables, the program's behavior becomes **unpredictable** due to the following factors:

- **Race conditions**: If two threads write to the same variable simultaneously, the outcome is uncertain— which thread's update prevails?
- **Interleaving execution**: The program's behavior depends on how threads interleave their access to shared data.
- **Non-deterministic scheduling**: Thread execution order is unpredictable and may change between program runs.
- **Interruptions**: External interrupts can occur at any time, further affecting execution flow.
- **Instruction reordering**: Compilers and hardware may **reorder instructions** for optimization, potentially breaking expected execution order (e.g., `c = a + b; x = y * y;` might not execute sequentially).
- **Atomicity limitations**: On modern systems, **only single-word aligned read/write operations** are guaranteed to be atomic—multi-word or multi-instruction operations are **not inherently atomic**.

Can This Code Cause a Panic?

Thread 1

```
p = someFn();
isInitialized = true;
```

Thread 2

```
while (!isInitialized);
q = aFn(p);
if (q != aFn(someFn()))
    panic;
```

Key Observations:

- `someFn()` and `aFn()` are **stateless functions** (they have no side effects).
- `Thread 2` waits for `isInitialized` to be `true` before proceeding.
- The panic occurs if `q != aFn(someFn())`.

Why Can This Panic?

Yes, this code **can panic** due to **instruction reordering** by the **compiler** or **processor**:

1. **Compiler Reordering:**
   - The compiler may **swap** the execution order of:

     ```
     isInitialized = true;
     p = someFn();
     ```

   - This means `isInitialized` could be set to `true` **before** `p` is assigned a valid value.
2. **Processor Reordering:**
   - Some architectures allow out-of-order execution for performance reasons.
   - The CPU might execute `isInitialized = true` **before** `p = someFn();`, leading `Thread 2` to read an **uninitialized** `p`.

Consequences:

If `Thread 2` reads `isInitialized` as `true` **before** `p` is actually set by `Thread 1`, it may pass an **invalid or stale value** of `p` to `aFn()`. This can cause `q != aFn(someFn())`, triggering a **panic**.

How to Fix This?

To ensure correct execution order, we need **memory barriers** or **synchronization mechanisms** like:

- **Atomic operations**
- **Mutexes/locks**

- **Volatile memory access**
- **Explicit memory fences (e.g., `std::atomic_thread_fence` in C++)**

Why Do Compilers Reorder Instructions?

Compilers **rearrange instructions** to optimize performance while preserving single-threaded correctness. This happens for several reasons:

- **Efficient Code Generation:**
  - The compiler analyzes **control** and **data dependencies** to minimize delays and improve execution speed.
- **Register and Instruction-Level Parallelism:**
  - Reordering can help **reduce stalls** in pipelines and improve parallel execution.
- **Multi-Threading Challenges:**
  - If variables can **spontaneously change** (e.g., due to another thread modifying shared memory), many compiler optimizations become **unsafe**.
  - Compilers assume single-threaded execution unless explicitly told otherwise (e.g., `volatile`, `std::atomic`).

Why Do CPUs Reorder Instructions?

Modern processors also reorder instructions at runtime for **performance reasons**:

- **Write Buffering:**
  - Instead of waiting for a **write to memory** to complete, the CPU allows subsequent instructions to execute.
- **Instruction Pipelines:**
  - To maximize efficiency, CPUs **dynamically reorder** operations, ensuring that instructions don't stall due to **memory accesses** or **dependencies**.

How to Prevent Unwanted Reordering?

To **ensure correct execution order**, use **Memory Barriers (Fences, WMB - Write Memory Barrier)**, which enforce strict ordering:

- **Memory Barrier Instructions:**
  - These force **all operations before the barrier** to complete before execution continues.
  - No operation **after** the barrier starts until the barrier completes.
- **Implicit Memory Barriers:**
  - Synchronization mechanisms such as **locks, mutexes, and atomic operations** include **built-in memory barriers** to prevent reordering.

Without Memory Barrier (Potential Issue)

```
#include <iostream>
#include <thread>

int x = 0, y = 0;
int a = 0, b = 0;

void thread1() {
    x = 1;      // Write to x
    a = y;      // Read from y
}

void thread2() {
    y = 1;      // Write to y
    b = x;      // Read from x
}

int main() {
    std::thread t1(thread1);
    std::thread t2(thread2);
    t1.join();
```

```
    t2.join();

    std::cout << "a = " << a << ", b = " << b << std::endl;
}
```

Possible Unexpected Output

Due to **instruction reordering**, we might observe:

```
a = 0, b = 0
```

even though it seems that `x = 1` and `y = 1` should be visible across threads.

With Memory Barrier (Safe Solution)

To enforce the correct execution order, we use **std::atomic** with explicit memory barriers.

```
#include <iostream>
#include <thread>
#include <atomic>

std::atomic<int> x(0), y(0);
std::atomic<int> a(0), b(0);

void thread1() {
    x.store(1, std::memory_order_release); // Ensures previou
s writes are visible
    a.store(y.load(std::memory_order_acquire), std::memory_or
der_relaxed);
}

void thread2() {
    y.store(1, std::memory_order_release);
    b.store(x.load(std::memory_order_acquire), std::memory_or
der_relaxed);
```

```
}

int main() {
    std::thread t1(thread1);
    std::thread t2(thread2);
    t1.join();
    t2.join();

    std::cout << "a = " << a.load() << ", b = " << b.load() <
< std::endl;
}
```

Why Does This Work?

- `std::memory_order_release` ensures that **all previous operations** complete before the store.
- `std::memory_order_acquire` ensures that **no subsequent operations** start before the load.
- Using **std::atomic** prevents the **compiler from reordering** instructions.

Too Much Milk Problem

Introduction

The **Too Much Milk Problem** is a common issue in the context of **concurrent programming** and the management of **shared resources**. It demonstrates how the lack of synchronization between processes or threads accessing the same resource can lead to problems like **race conditions**, **deadlock**, or **resource contention**.

In this scenario, two entities (often threads or processes) share a resource (in this case, milk) and need to coordinate to avoid conflicts.

Scenario

Imagine two people, A and B, who live together. The milk is finished, so both decide to go out and buy some. If there's no coordination mechanism between them, both might end up buying milk, creating an excess of milk in the house. This represents the **Too Much Milk Problem**.

Objective

The goal is to ensure that **only one** of the two people buys the milk, avoiding:

- **Duplication**: both buy the milk.
- **Absence of purchase**: neither buys the milk.

The Problem in Detail

The problem occurs when two or more threads/processes attempt to access and modify a shared resource without an adequate synchronization mechanism. In this case, the shared resource is the milk (a variable that indicates whether milk is available or not).

If both threads/processes access the variable simultaneously, the following issues may arise:

- **Race Condition**: both threads see that milk is finished and decide to buy it, leading to two purchases.
- **Deadlock**: both threads might try to acquire a lock on a shared resource in an order that locks them mutually, causing them to be stuck.

Wrong Solutions

1. No Synchronization (Race Condition)

```
if (noMilk) {
    buyMilk();
}
```

Problem: Both threads might see that the milk is finished and both execute `buyMilk()`, resulting in two purchases.

2. Using a "Note" (Naive Attempt)

```
if (noMilk && !note) {
    leaveNote();
    buyMilk();
    removeNote();
}
```

Problem: Even if we use a "note" to synchronize access, both threads might still decide to buy the milk at the same time. The race condition is not prevented.

Correct Solution: Mutual Exclusion (Mutex)

The correct solution to the **Too Much Milk Problem** is the use of **mutex** (mutual exclusion). A mutex is a mechanism that ensures only one thread can access a shared resource at a time.

3. Solution with Mutex (Locking)

The following code uses a **mutex** to ensure that only one thread can check and buy the milk at a time.

```
#include <mutex>

std::mutex mtx;  // Lock

void buyMilkIfNeeded() {
    mtx.lock();  // Acquires the lock
    if (noMilk) {
        buyMilk();  // Buy the milk only if it's not availabl
e
    }
    mtx.unlock();  // Releases the lock
}
```

How It Works:

- Each time a thread wants to buy the milk, it must first acquire the lock (`mtx.lock()`).

- If another thread has already acquired the lock, the other thread will have to wait until the lock is released (`mtx.unlock()`).
- This ensures that only one thread can enter the critical section and buy the milk.

Benefits of the Mutex Solution

- **Avoids race condition**: Only one thread can access the shared resource (milk) at a time.
- **Prevents deadlock**: The code never gets stuck permanently because the lock is correctly released.
- **Improved performance**: Compared to other solutions like busy-waiting, mutexes are more efficient as they do not waste CPU resources.

Alternatives and Improvements

- **Are you sure the lock is always released?** Use `std::lock_guard` in C++ to automatically manage the release of the lock:

```
#include <mutex>

std::mutex mtx;

void buyMilkIfNeeded() {
    std::lock_guard<std::mutex> lock(mtx);  // Lock is automa
tically
                                            //released when t
he function exits
    if (noMilk) {
        buyMilk();
    }
}
```

- **Deadlock Handling**: In more complex scenarios with multiple shared resources, it's important to design the code to acquire locks in a defined order to prevent deadlock.

Race Condition

A **race condition** occurs when the behavior of a concurrent program depends on the **order of execution** of operations across multiple threads. In particular:

- **Order of operations**: The final outcome of the program depends on the interleaving (i.e., the scheduling order) of the threads.
- **Write operations**: At least one of the operations involved in the race condition must be a **write** to shared data. This can cause inconsistent or unexpected results.

Example: Two threads try to update a shared bank account balance. Without synchronization, both threads may read the balance, add money, and then write the new balance back, leading to incorrect results.

```
#include <iostream>
#include <thread>

int balance = 0;

void deposit(int amount) {
    int temp = balance;  // Read the balance
    temp += amount;      // Add the amount
    balance = temp;      // Write the new balance
}

int main() {
    std::thread t1(deposit, 100);
    std::thread t2(deposit, 200);

    t1.join();
    t2.join();

    std::cout << "Final balance: " << balance << std::endl;
```

```
// Unexpected result
    return 0;
}
```

Problem: If both threads read the balance before one writes the new value, the final balance could be incorrect. For example, both might read 0 and add 100 and 200, respectively, resulting in a final balance of 200 instead of 300.

Mutual Exclusion

Mutual exclusion is the principle that ensures **only one thread** can execute a particular piece of code (or access a shared resource) at a time. This is crucial for preventing concurrent threads from interfering with each other when accessing shared data, leading to unpredictable behavior.

Example: Using a **mutex** to protect the bank account balance.

```
#include <iostream>
#include <thread>
#include <mutex>

int balance = 0;
std::mutex mtx;

void deposit(int amount) {
    std::lock_guard<std::mutex> lock(mtx);  // Acquire the lo
ck
    int temp = balance;
    temp += amount;
    balance = temp;
}

int main() {
    std::thread t1(deposit, 100);
    std::thread t2(deposit, 200);

    t1.join();
```

```
    t2.join();

    std::cout << "Final balance: " << balance << std::endl;
// Correct result
    return 0;
}
```

Benefit: Mutual exclusion ensures that only one thread can access `balance` at a time, preventing race conditions and ensuring the final balance is correct (300).

Critical Section

A **critical section** refers to a part of the code that should be executed by **only one thread at a time**. It's the part of the program where shared resources (such as variables, data structures, etc.) are accessed or modified. Without proper synchronization, multiple threads could enter the critical section simultaneously, leading to issues like race conditions.

Example: Protecting a critical section with a mutex.

```
#include <iostream>
#include <thread>
#include <mutex>

int shared_data = 0;
std::mutex mtx;

void increment() {
    std::lock_guard<std::mutex> lock(mtx);  // Protect the cr
itical section
    shared_data++;  // Critical section
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);
```

```
    t1.join();
    t2.join();

    std::cout << "Shared data: " << shared_data << std::endl;
// Correct result
    return 0;
}
```

Benefit: The critical section is protected by the mutex, so only one thread can increment `shared_data` at a time, preventing unintended behavior like double increments

Lock

A **lock** is a synchronization mechanism used to enforce mutual exclusion. It acts as a control to ensure that only one thread can access the critical section at a time. Here's how it works:

- **Lock before entering**: A thread must acquire the lock before it can enter the critical section. This ensures no other thread can access the shared resource while it is being used.
- **Unlock after leaving**: Once the thread finishes its operation within the critical section, it releases the lock so other threads can acquire it and access the resource.
- **Wait if locked**: If the lock is already acquired by another thread, the requesting thread will have to wait until the lock becomes available. Waiting can either be:
  - **Active waiting** (spinning in a loop while checking the lock)
  - **Passive waiting** (sleeping or blocking until the lock is available)

Example: Using a lock to protect a shared resource.

```
#include <iostream>
#include <thread>
#include <mutex>
```

```
std::mutex mtx;

void critical_section(int id) {
    mtx.lock();  // Acquire the lock
    std::cout << "Thread " << id << " entering critical secti
on.\n";
    // Perform operations on the shared resource
    std::cout << "Thread " << id << " leaving critical sectio
n.\n";
    mtx.unlock();  // Release the lock
}

int main() {
    std::thread t1(critical_section, 1);
    std::thread t2(critical_section, 2);

    t1.join();
    t2.join();

    return 0;
}
```

Benefit: Using the lock, we ensure that only one thread can enter the critical section at a time, preventing interference between threads.

Rules for using Lock

1. Lock is Initially Free

- **The lock should be available (unlocked)** when the program starts. The thread can acquire it when needed, but it should not be locked by default.

  Example:

  ```
  std::mutex mtx;  // Initially unlocked
  ```

2. Always Acquire the Lock Before Accessing Shared Data Structures

- **Acquire the lock before accessing any shared resource** or shared data structure. This ensures that no other thread can access the resource simultaneously, preventing race conditions.

  Example:

```
void update_shared_data() {
    std::lock_guard<std::mutex> lock(mtx);  // Acquire the lock before accessing shared data
    shared_data++;  // Safe access to shared data
}
```

  Important: Lock acquisition should always happen at the **beginning** of the procedure or critical section, to ensure no other thread can interfere.

3. Always Release the Lock After Finishing with Shared Data

- **Release the lock after you finish using the shared resource**. This ensures that other threads can access the resource once your thread is done.

  Example:

```
void update_shared_data() {
    std::lock_guard<std::mutex> lock(mtx);  // Acquire lock
    shared_data++;  // Access shared data
    // Lock is automatically released when lock goes out of scope
}
```

  DO NOT throw the lock for someone else to release. Always release the lock within the same thread that acquired it to avoid confusion and errors.

4. Never Access Shared Data Without Lock

- **Never access shared data without holding the appropriate lock**. Doing so can lead to a **race condition**, where multiple threads manipulate the shared resource concurrently, leading to inconsistent or undefined behavior.

  Example:

  ```
  // Incorrect: accessing shared data without a lock can cau
  se race conditions
  shared_data++;  // Potential race condition if accessed co
  ncurrently without a lock
  ```

  Always ensure that the critical section is protected by the lock:

  ```
  std::lock_guard<std::mutex> lock(mtx);  // Correct: Lock a
  cquired before accessing shared data
  shared_data++;  // Safe access
  ```

---

5. Do Not Share Objects on the Stack

- **Avoid sharing stack-allocated objects** between threads. Stack-allocated variables are destroyed when the function scope ends, which can cause undefined behavior if another thread tries to access that object.

  Example:

  ```
  void thread_function() {
      std::mutex mtx;  // Stack-allocated mutex (avoid shari
  ng)
     // This mutex will be destroyed when thread_function ex
  its, causing issues
  }
  ```

  Instead, allocate shared objects on the heap to ensure they persist beyond the scope of individual functions:

```
void thread_function() {
    std::shared_ptr<std::mutex> mtx = std::make_shared<st
d::mutex>();
    // Heap-allocated mutex
    // This mutex persists and can be shared safely betwee
n threads
}
```

Additional Best Practices

- **Avoid locking large sections of code**: Try to minimize the critical section to reduce contention between threads. Only lock what's necessary for accessing shared data.
- **Prefer lock guards or unique locks**: Use `std::lock_guard` or `std::unique_lock` for automatic lock management. These classes automatically release the lock when they go out of scope, reducing the risk of forgetting to release it.

  Example:

  ```
  void update_shared_data() {
      std::lock_guard<std::mutex> lock(mtx);  // RAII-style
  lock
      shared_data++;  // Safe access
  }
  ```

Producer - Consumer problem

The **Producer-Consumer problem** is a classic synchronization problem that deals with coordinating two types of processes, producers and consumers, that share a common resource, typically a **buffer** or a **queue**.

The challenge is to ensure that the producers and consumers operate efficiently and safely in a concurrent environment, without running into problems like race conditions, data corruption, or deadlocks.

Problem Overview

- **Producers** generate data, items, or tasks, and put them into a shared buffer (or queue).
- **Consumers** take the items from the buffer and process them.
- The buffer has a limited size, and both producers and consumers may need to wait when the buffer is either full (producers cannot add more items) or empty (consumers cannot take more items).

Challenges

1. **Synchronization**: Producers and consumers need to coordinate their operations to avoid race conditions where the buffer is accessed simultaneously by multiple threads.
2. **Buffer Overflow**: The producer must wait if the buffer is full.
3. **Buffer Underflow**: The consumer must wait if the buffer is empty.
4. **Efficiency**: The system should be efficient, avoiding unnecessary waiting while ensuring safe data access.

Solution Using Locks and Condition Variables

The solution typically involves using **mutual exclusion** (mutexes) and **condition variables** to synchronize the producer and consumer threads. The mutex ensures that only one thread (either a producer or a consumer) can access the buffer at a time, while the condition variable allows the threads to wait for certain conditions (e.g., waiting for the buffer to have space or items).

Algorithm and Explanation

Here’s how we can break down the producer-consumer solution:

1. **Producer**:
    - Wait until the buffer has space.
    - Add an item to the buffer.
    - Signal the consumer (if necessary) that there is now an item to consume.
2. **Consumer**:

- Wait until the buffer has an item.
- Remove an item from the buffer.
- Signal the producer (if necessary) that there is now space available in the buffer.

Code Example (C++ with Condition Variables)

Here's an example implementation of the producer-consumer problem using **C++** with **mutexes** and **condition variables**:

```
#include <iostream>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>

std::queue<int> buffer;  // Shared buffer
const int BUFFER_SIZE = 5;
std::mutex mtx;
std::condition_variable cv_empty, cv_full;  // Condition vari
ables for empty and full buffer

// Producer function
void producer() {
    int item = 0;
    while (true) {
        std::this_thread::sleep_for(std::chrono::milliseconds
(100));  // Simulate production time

        std::unique_lock<std::mutex> lock(mtx);
        cv_full.wait(lock, []{ return buffer.size() < BUFFER_
SIZE; });  // Wait if the buffer is full

        // Produce an item and add it to the buffer
        buffer.push(item);
        std::cout << "Produced: " << item << std::endl;
```

```
        item++;

        // Notify the consumer that an item is available
        cv_empty.notify_one();
    }
}

// Consumer function
void consumer() {
    while (true) {
        std::this_thread::sleep_for(std::chrono::milliseconds
(200));  // Simulate consumption time

        std::unique_lock<std::mutex> lock(mtx);
        cv_empty.wait(lock, []{ return !buffer.empty(); });
// Wait if the buffer is empty

        // Consume an item from the buffer
        int item = buffer.front();
        buffer.pop();
        std::cout << "Consumed: " << item << std::endl;

        // Notify the producer that space is available in the
buffer
        cv_full.notify_one();
    }
}

int main() {
    // Start the producer and consumer threads
    std::thread producer_thread(producer);
    std::thread consumer_thread(consumer);

    // Wait for both threads to finish (in a real applicatio
n, this would likely run indefinitely)
    producer_thread.join();
```

```
    consumer_thread.join();

    return 0;
}
```

Explanation of Code:

1. **Shared Buffer**: A `std::queue<int>` is used as the shared buffer, where the producer puts items and the consumer takes them.
2. **Mutex (`mtx`)**: Ensures that only one thread can access the buffer at a time.
3. **Condition Variables**:
   - `cv_empty` is used to notify the consumer when the buffer is not empty (i.e., there's an item to consume).
   - `cv_full` is used to notify the producer when there's space in the buffer (i.e., the buffer isn't full).
4. **Producer**: Produces an item and waits if the buffer is full. After adding an item, it notifies the consumer.
5. **Consumer**: Consumes an item and waits if the buffer is empty. After consuming an item, it notifies the producer.
6. **Thread Synchronization**: The producer and consumer threads run concurrently, and they coordinate using the mutex and condition variables to avoid race conditions and to manage buffer overflows/underflows.

Key Concepts:

- **Condition Variables**: These are used to efficiently block a thread until some condition is met (e.g., buffer is not full or empty). They avoid busy waiting (active waiting), which wastes CPU cycles.
- **Mutexes**: These ensure that only one thread can access the shared buffer at a time, preventing race conditions where multiple threads modify the buffer simultaneously.

Scalability:

- The producer-consumer problem can be scaled to support multiple producers and multiple consumers by adjusting the synchronization logic and ensuring proper coordination between threads.

Condition Variables in Synchronization

Condition variables are synchronization primitives that are used to block a thread until a certain condition is met. They provide a way for threads to **wait** for specific events to occur, and they allow threads to **notify** others when those events happen.

In the context of the **Producer-Consumer problem** or other concurrent systems, condition variables are often used to coordinate threads based on the state of shared resources.

Let's break down the important aspects of condition variables based on your points:

1. ALWAYS Hold Lock When Calling `wait()` (and also `signal()` / `broadcast()`)

Condition variables are designed to provide synchronization for shared state. This means that they must be used in conjunction with a **mutex** or **lock** to ensure that shared data is accessed safely.

Why Do We Need to Hold the Lock?

- **Consistency**: The shared data (e.g., buffer in the producer-consumer problem) needs to be in a consistent state when the thread calls `wait()` or signals others. Holding the lock ensures that no other thread can modify the shared data while the current thread is waiting or signaling.
- **Preventing Race Conditions**: Without holding the lock, multiple threads could simultaneously modify the shared state or condition, leading to race conditions.

Thus, it's **mandatory** to acquire the lock before interacting with a condition variable.

```
std::unique_lock<std::mutex> lock(mtx); // Acquire the lock
cv.wait(lock);  // Wait until the condition is met
```

Similarly, `signal()` or `broadcast()` must also be called while holding the lock, ensuring that the shared state is consistent when notifying waiting threads.

2. Condition Variables Provide Synchronization FOR Shared State

A condition variable **does not** by itself manage synchronization of shared resources. Rather, it is used to signal when the **state** of shared resources has changed in a way that requires other threads to act.

- Condition variables let threads wait for a certain condition (like buffer being full or empty) to be true before proceeding.
- **Shared state** (such as the buffer) must be protected by a **mutex** to ensure that no two threads access the shared resource concurrently.

3. A Condition Variable is Memoryless

This means that **the condition variable does not remember its state**. It is simply a signaling mechanism that tells a waiting thread to proceed once the condition has been met.

- **If a signal occurs when no thread is waiting**, the signal does nothing (it's a no-op).
- **If a thread is waiting and another signals**, the waiting thread will be notified and will resume execution.

This behavior implies that condition variables do not retain any information about whether a thread is waiting or not. They are "memoryless" because they only act when a thread is waiting and the signal is triggered.

4. `wait()` Atomically Releases Lock

When a thread calls `wait()` on a condition variable, **it releases the lock** and blocks until another thread signals it.

What Happens When a Thread Calls `wait()`?

- The calling thread **atomically** releases the lock and waits for the signal.
- This atomic release is important because it ensures that the thread doesn't hold the lock while it's waiting. It prevents deadlocks and allows other threads to acquire the lock and modify shared resources.

```
cv.wait(lock); // Atomically releases the lock and waits
```

The lock is automatically reacquired when the thread is woken up from the `wait()` call. This ensures that the thread can safely proceed and access shared data.

What If `wait()`, Then Release the Lock?

Releasing the lock **after** calling `wait()` would result in incorrect behavior. Since `wait()` already releases the lock, releasing the lock afterward would lead to a **double unlock**, which is an error.

What If Release the Lock, Then `wait()`?

This scenario is also problematic. Releasing the lock before calling `wait()` could cause a **race condition**, as another thread might change the shared state before the current thread is waiting. This could cause the thread to miss the signal or wake up prematurely.

Correct Use of `wait()` and `signal()` / `broadcast()`

1. Call `wait()` While Holding the Lock

The correct way to use `wait()` is:

1. Acquire the lock.
2. Call `wait()` to block the thread while it is holding the lock. This allows other threads to acquire the lock and modify the shared state.
3. Once the condition is met and the thread is notified, the lock is automatically reacquired.

Example:

```
std::unique_lock<std::mutex> lock(mtx);
while (buffer_is_empty()) { // Check condition in the loop
    cv.wait(lock); // Atomically releases lock and waits for
condition
}
// Proceed with buffer processing
```

2. Signal or Broadcast While Holding the Lock

When a thread is ready to signal other threads (either by calling `notify_one()` or `notify_all()`), it must hold the lock. This ensures that the shared state is properly updated and that waiting threads are notified while the shared state is consistent.

Example:

```
std::unique_lock<std::mutex> lock(mtx);
// Update shared state
cv.notify_one(); // Notify one waiting thread
// or
cv.notify_all(); // Notify all waiting threads
```

Structured Synchronization in Concurrent Programming

Structured synchronization is a key concept in concurrent programming, especially when dealing with shared resources that may be accessed by multiple threads.

The goal is to ensure that shared data is protected from race conditions and that threads can safely coordinate with one another. Below, we'll break down the essential principles of structured synchronization.

---

1. Identify Shared Objects or Data Structures

The first step in structured synchronization is to **identify** the objects or data structures that will be accessed by multiple threads concurrently.

These shared resources could be anything from buffers, queues, or lists, to more complex structures like databases or in-memory caches.

Examples:

- A **bounded buffer** in the producer-consumer problem.
- A **counter** or **variable** that multiple threads update.
- A **data structure** (e.g., a linked list or tree) shared by multiple threads.

2. Add Locks to the Object/Module

Once shared resources are identified, you need to ensure that only one thread can access these resources at a time. This is done by adding locks to protect critical sections, i.e., parts of the code that modify shared data.

Steps:

- **Grab the lock at the start** of any method or procedure that will access shared resources.
- **Release the lock at the end** of the method/procedure once the work is done.

Example in C++:

```
std::mutex mtx;

void updateSharedResource() {
    std::lock_guard<std::mutex> lock(mtx); // Acquiring lock
    // Access and modify shared resource
    sharedData++;
    // Lock is automatically released at the end of the scope
}
```

By using `std::lock_guard` (or similar mechanisms like `std::unique_lock`), you ensure that the lock is released when the function scope ends, even if an exception occurs.

3. Handle Waiting

Sometimes, a thread needs to wait for a condition to be met before proceeding. This can occur when the thread cannot proceed because the shared state is not in the desired condition (for example, a consumer waits for a producer to put data into the buffer).

Condition Variables for Waiting:

- Use a **while loop** with `condition.wait(lock)` to repeatedly check the condition while waiting.
- The `wait` method will atomically release the lock, allowing other threads to acquire it while the current thread is waiting.

Example in C++:

```
std::condition_variable cv;
std::mutex mtx;
bool isDataReady = false;

void consumer() {
    std::unique_lock<std::mutex> lock(mtx); // Acquiring lock
    while (!isDataReady) {
        cv.wait(lock); // Wait while condition is not met
    }
    // Process the shared data once ready
}
```

Here, the `consumer` thread will wait for `isDataReady` to become `true`, and it will release the lock and block while waiting. When the condition changes, the thread will be notified and will re-acquire the lock to proceed.

4. Signaling or Broadcasting (Waking Up Threads)

When a thread changes the shared state in such a way that another thread could proceed, it needs to notify the waiting threads. You can use `signal()` (notify one thread) or `broadcast()` (notify all waiting threads).

- **Prefer** `signal()`: In most cases, signaling only one waiting thread is sufficient. Broadcasting is only necessary when multiple threads might be waiting and need to be awakened.

Signal Example:

```
void producer() {
    std::unique_lock<std::mutex> lock(mtx);
    // Modify shared state
    isDataReady = true;
    cv.notify_one(); // Notify one waiting thread
}
```

Broadcast Example:

```
void producer() {
    std::unique_lock<std::mutex> lock(mtx);
    // Modify shared state
    isDataReady = true;
    cv.notify_all(); // Notify all waiting threads
}
```

- **Be cautious**: Do not assume which thread will be executed when signaled. The thread scheduler may choose any thread, and the thread that is woken up might still need to re-check the condition before proceeding.

5. Ensure Consistent Shared State

It's essential to **always leave shared state variables in a consistent state** when releasing the lock or when waiting. If a thread is modifying shared data, it should do so in such a way that, after releasing the lock, the shared state remains logically consistent and safe for other threads to use.

When Releasing the Lock or Waiting:

- **Ensure that the shared data is in a valid and consistent state**. If necessary, perform checks or maintain invariants to avoid corrupting shared resources.

- **Avoid partial updates** of shared data—updates should be atomic, meaning that they should either complete entirely or not happen at all.

Example:

```
void safeUpdateSharedData() {
    std::lock_guard<std::mutex> lock(mtx); // Acquire lock
    // Ensure that shared data remains consistent
    if (sharedData < 0) {
        sharedData = 0; // Correct any inconsistency
    }
    sharedData++; // Perform the update
    // Lock is released automatically at the end of the scope
}
```

In this example, the lock ensures that the `sharedData` is not accessed concurrently, and the state remains consistent throughout the update process.

Summary of Structured Synchronization Rules

1. **Identify shared objects**: Determine which data structures are shared among threads.
2. **Add locks**: Use locks to protect access to shared resources, ensuring that only one thread can access them at a time.
3. **Wait for conditions**: Use condition variables and a `while` loop with `wait()` to wait for specific conditions to be met, releasing the lock while waiting.
4. **Signal or Broadcast**: Use `notify_one()` or `notify_all()` to wake up waiting threads when the condition changes. Prefer signaling one thread to minimize unnecessary context switches.
5. **Ensure consistent state**: Always ensure that shared state remains consistent before releasing the lock or waiting, avoiding race conditions and ensuring safe thread synchronization.

Mesa vs. Hoare Semantics

In concurrent programming, particularly when dealing with **condition variables** and **synchronization**, two different types of semantics are used to describe how threads interact with condition variables: **Mesa semantics** and **Hoare semantics**.

These semantics define how signaling and waiting threads interact with locks and processor control. Both are used to control the flow of execution in multithreaded programs, but they differ in their behavior and the order in which threads are executed after being signaled.

---

1. Mesa Semantics (Used in Hansen's textbook)

Mesa semantics is the more **general and widely used** approach. It follows the following principles:

- **Signal puts the waiter on the ready list**: When a thread signals a condition variable, it places the waiting thread on a **ready list** (i.e., it indicates that a waiting thread can now be scheduled to run).
- **Signaler retains the lock and processor**: The thread that signals the condition variable **keeps the lock** and the processor. It can continue execution until it explicitly releases the lock.
- **Thread scheduling**: After signaling, the signaled thread will not immediately execute. Instead, the signaling thread must release the lock before the signaled thread can be scheduled and executed.

Key Features of Mesa Semantics:

- The signaled thread **may not immediately execute**, since the signaler's thread continues executing while the waiting thread is only moved to the ready list.
- There is no guarantee of **immediate execution** of the signaled thread, as it will only be executed when the scheduler allows.

Example of Mesa Semantics:

```
std::mutex mtx;
std::condition_variable cv;
bool isDataReady = false;

void producer() {
```

```
    std::unique_lock<std::mutex> lock(mtx);
    // Produce data
    isDataReady = true;
    cv.notify_one();  // Signal that data is ready, but produ
cer keeps the lock
}

void consumer() {
    std::unique_lock<std::mutex> lock(mtx);
    while (!isDataReady) {
        cv.wait(lock);  // Wait for data to be produced
    }
    // Consume data
}
```

In this example, after the producer signals the condition variable, it **continues to hold the lock**, and the consumer will not execute until the producer releases the lock and the thread scheduler grants the consumer CPU time.

---

2. Hoare Semantics

Hoare semantics is a **more strict and idealized** approach that gives **stronger guarantees** when signaling and waiting for threads:

- **Signal gives processor and lock to waiter**: When a thread signals a condition variable, **it gives up the processor** and the **lock is handed over** to the waiting thread. The signaled thread will execute immediately after being signaled.
- **State is guaranteed unchanged**: The thread that is signaled may assume that the state of shared variables has not changed since the signal was issued, allowing it to safely resume its execution without checking the condition again.
- **When the waiter finishes**: Once the waiter finishes executing, the lock and processor are handed back to the signaling thread.
- **Nested signals**: In Hoare semantics, **nested signals** are possible. That is, a thread may signal another thread while already holding a lock, allowing for more precise control over thread execution.

Key Features of Hoare Semantics:

- The **signaled thread immediately starts execution** once it is woken up, and the signaling thread gives up both the **processor** and the **lock**.
- The signaled thread can proceed with the assumption that the state of shared resources has **not changed** between the signal and its execution.
- **Nested signals** can be used, where a thread can signal another while still executing within a critical section.

Example of Hoare Semantics:

```
std::mutex mtx;
std::condition_variable cv;
bool isDataReady = false;

void producer() {
    std::unique_lock<std::mutex> lock(mtx);
    // Produce data
    isDataReady = true;
    cv.notify_one();  // Signal the consumer and immediately
relinquish lock
}

void consumer() {
    std::unique_lock<std::mutex> lock(mtx);
    while (!isDataReady) {
        cv.wait(lock);  // Wait for data to be produced
    }
    // Consume data
}
```

In this case, after the producer signals the condition variable, **the producer gives up the lock and CPU**, and the consumer **immediately gets scheduled** to execute. This ensures that the consumer can safely assume the data has not been changed between the signal and its execution.

3. The Producer-Consumer Problem with Bounded Buffer

In the context of the **Producer-Consumer** problem, a **bounded buffer** is often used to store data produced by the producer threads and consumed by the consumer threads.

- The bounded buffer allows **producers** to add data to the buffer until it's full, and **consumers** to remove data from the buffer until it's empty.
- The solution works with **both Mesa** and **Hoare semantics**, but it does **not make assumptions** about the order in which:
  - The producers that are waiting on the condition variable are woken up to deposit their messages.
  - The consumers that are waiting are woken up to consume data.

This means that whether the **Mesa** or **Hoare** model is used, the exact order in which waiting producers and consumers are awakened can vary. In both cases, the synchronization mechanism ensures that only one thread at a time can access the buffer, and that the system will function as expected without race conditions or deadlocks.

Important Notes:

- In **Mesa semantics**, the order of producers or consumers being woken up is not guaranteed, as the signaler retains the lock.
- In **Hoare semantics**, the first signaled thread will immediately be woken up and can execute right after being signaled, but again, there is no guarantee about the order of execution for waiting threads.

Summary of Differences Between Mesa and Hoare Semantics

Feature	**Mesa Semantics**	**Hoare Semantics**
Signal Behavior	Places the waiter on the ready list, signaller keeps lock	Signaller gives processor and lock to the waiter
Thread Execution	Signaled thread may not execute immediately	Signaled thread executes immediately

State Assumptions	Signaled thread may not assume state is unchanged	Signaled thread can assume state is unchanged
Lock Handling	Signaler retains lock after signaling	Signaler gives up the lock after signaling
Nested Signals	Not typically supported	Nested signals are possible

Both approaches work well for the **bounded buffer** solution in the Producer-Consumer problem, but their differences influence how the threads are scheduled and how the shared state is managed in a multithreaded environment.

FIFO Bounded Buffer (Hoare Semantics)

In the **Hoare semantics** model, the interaction between threads is stricter than in Mesa semantics, as the signaling thread gives both the processor and the lock to the signaled thread. The following example describes the behavior of a **FIFO bounded buffer** with **Hoare semantics**, where `get()` and `put()` operations are implemented with condition variables and locks.

Hoare Semantics - FIFO Bounded Buffer

`get()` (Consumer)

```
get() {
    lock.acquire();  // Acquire the lock to access the shared
buffer
    while (nelem == 0)  // If buffer is empty
        empty.wait(&lock);  // Wait until there is at least o
ne item to consume

    item = buf[front];  // Get the item from the front of the
buffer
    front = (front + 1) % size;  // Move the front pointer in
a circular manner
    nelem--;  // Decrease the number of elements in the buffe
r
```

```
    full.signal(&lock);  // Signal the producer that there is
space
                                                // available i
n the buffer
    lock.release();  // Release the lock after modifying shar
ed data

    return item;  // Return the item that was consumed
}
```

`put()` (Producer)

```
put(item) {
    lock.acquire();  // Acquire the lock to access the shared
buffer
    while (nelem == size)  // If buffer is full
        full.wait(&lock);  // Wait until there is space avail
able in the buffer

    buf[last] = item;  // Place the item in the buffer at the
"last" position
    last = (last + 1) % size;  // Move the last pointer in a
circular manner
    nelem++;  // Increase the number of elements in the buffe
r

    empty.signal(&lock);  // Signal the consumer that there i
s an item available
    lock.release();  // Release the lock after modifying shar
ed data
}
```

Explanation of Hoare Semantics Behavior

- **Condition Variables** (`empty` and `full`): These condition variables represent the states of the buffer. `empty` is used by consumers to wait when the buffer is empty, while `full` is used by producers to wait when the buffer is full.
- **FIFO Queue**: In Hoare semantics, the thread that signals the condition variable gives both the processor and the lock to the waiting thread. This guarantees that the waiting thread (the consumer in `get()` or the producer in `put()`) will execute immediately after being signaled. This thread can safely assume that the state of the buffer has not changed since the signal was issued.

Key Notes:

- The **signaled thread** immediately gets the lock and the CPU, allowing it to proceed without waiting.
- The **FIFO behavior** is maintained by the condition variables, which keep the threads in the order they were signaled.

FIFO Bounded Buffer (Mesa Semantics)

The same FIFO bounded buffer implementation can be adapted for **Mesa semantics**, but there are key differences in the way the waiting threads interact with the condition variables.

`get()` (Consumer)

```
get() {
    lock.acquire();  // Acquire the lock to access the shared
buffer
    if (nextGet.empty() || nelem == 0) {  // If no consumers
waiting or

//   the buffer is empty

        self = createCondition();  // Create a new condition
variable for the

//  waiting consumer
```

```
        nextGet.append(self);  // Add the consumer's conditio
n variable to the
                                                        //  waiti
ng list

        self.wait(lock);  // Wait until it is safe to proceed
                                                      // (i.e., unt
il there is data to consume)

        nextGet.remove(self);  // Remove the condition variab
le from the waiting
                                                         // list
after waking up

        destroyCondition(self);  // Clean up the condition va
riable
    }

    item = buf[front];  // Get the item from the front of the
buffer
    front = (front + 1) % size;  // Move the front pointer in
a circular manner
    nelem--;  // Decrease the number of elements in the buffe
r

    if (!nextPut.empty())  // If there are producers waiting
        nextPut.first()->signal();  // Signal the first produ
cer in

// the queue to wake up

    lock.release();  // Release the lock after modifying shar
ed data
```

```
    return item;  // Return the consumed item
}
```

`put()` (Producer)

```
put(item) {
    lock.acquire();  // Acquire the lock to access the shared
buffer
    while (nelem == size)  // If the buffer is full
        full.wait(&lock);  // Wait until there is space avail
able in the buffer

    buf[last] = item;  // Place the item in the buffer at the
"last" position
    last = (last + 1) % size;  // Move the last pointer in a
circular manner
    nelem++;  // Increase the number of elements in the buffe
r

    empty.signal(&lock);  // Signal the consumer that there i
s an item available
    lock.release();  // Release the lock after modifying shar
ed data
}
```

Explanation of Mesa Semantics Behavior

- **Condition Variables** (`nextGet` and `nextPut`): These are FIFO queues that store the condition variables for the threads waiting for the `get()` and `put()` operations, respectively.
  A producer or consumer waits by appending its condition variable to the appropriate queue (`nextGet` or `nextPut`), and signals the next thread in the queue when it finishes its operation.
- **Waiting Thread Behavior**: In Mesa semantics, when a thread calls `wait()` on a condition variable, it releases the lock and the signaling thread retains the

lock.
The signaled thread will not immediately execute; instead, it will only proceed once the lock is available and the condition it was waiting on is met.

- **Signal Behavior**: The signal does not immediately execute the waiting thread in Mesa semantics. Instead, the signal only places the waiting thread in the ready list, and the thread scheduler will decide when the thread is allowed to execute.

Key Notes:

- The **consumer thread** must explicitly check if there is a consumer waiting (by checking `nextGet`). If the buffer is empty, it creates a condition variable for itself and waits on it.
- After the consumer consumes an item, it checks if any producer is waiting (`nextPut` queue) and signals the first producer in the queue.
- In Mesa semantics, there is no guarantee that the signaled thread will execute immediately. The signaling thread releases the lock, allowing other threads to be scheduled.

Summary of Differences

Feature	**Hoare Semantics**	**Mesa Semantics**
Signal Behavior	Signal immediately gives the lock and processor to the waiting thread	Signal only places the waiting thread on a ready list
Thread Execution	Signaled thread executes immediately	Signaled thread waits until it gets the lock again
State Assumptions	Signaled thread assumes state is unchanged	Signaled thread may not assume state is unchanged until it gets the lock
Waiting Threads	No need to explicitly check for waiting threads	Waiting threads are managed in queues (e.g., `nextGet`, `nextPut`)

Implementing Synchronization: Lock Implementation with Software and Hardware Support

In a uniprocessor system, synchronization mechanisms are essential for controlling concurrent access to shared resources. Let's break down the implementation of a simple lock mechanism using different approaches:

Take 1: Software Implementation Using Memory Load/Store (e.g., Peterson's Algorithm)

Peterson's algorithm is an example of a software-based solution to achieve mutual exclusion. It uses memory operations (load and store) to synchronize the processes trying to access a critical section.

Here's a simplified overview of how it works:

- Two processes want to access a shared resource. They each have a flag indicating their intent to enter the critical section.
- One process signals that it's interested in accessing the critical section by setting its flag.
- The algorithm ensures that only one process enters the critical section at a time by using a shared variable that indicates whose turn it is.

```
bool flag[2] = {false, false};  // Two flags for two processe
s
int turn;  // Shared variable to decide whose turn it is

void enter_critical_section(int i) {
    flag[i] = true;  // Process i wants to enter
    turn = 1 - i;  // Set turn to the other process
    while (flag[1 - i] && turn == 1 - i) {
        // Wait for the other process to leave the critical s
ection
    }
}

void exit_critical_section(int i) {
    flag[i] = false;  // Process i has left the critical sect
```

```
ion
}
```

This solution ensures mutual exclusion without needing hardware support, but it assumes that memory operations (like `load` and `store`) are atomic and will not be interrupted.

Take 2: Lock Using Interrupt Disabling

In a uniprocessor system, disabling interrupts temporarily is a simple approach to ensure mutual exclusion. This works because the system can't switch to another process while interrupts are disabled, ensuring that no other process can access the shared resource during the critical section.

Lock Acquire with Interrupt Disabling

```
void lock_acquire(lock_t *L) {
    disableInterrupts();  // Disable interrupts to prevent co
ntext switches
    if (L->value == BUSY) {
        L->waiting.add(myTCB);  // Add the current thread to
the waiting queue
        suspend();  // Suspend the current thread, invoking c
ontext switch
    } else {
        L->value = BUSY;  // Lock the resource
    }
    enableInterrupts();  // Re-enable interrupts
}
```

Lock Release with Interrupt Disabling

```
void lock_release(lock_t *L) {
    disableInterrupts();  // Disable interrupts to ensure ato
mic operations
    if (!L->waiting.empty()) {
```

```
        thTCB = L->waiting.remove();  // Remove a thread from
the waiting list
        readyList.append(thTCB);  // Add the waiting thread t
o the ready list
    } else {
        L->value = FREE;  // Release the lock if no threads a
re waiting
    }
    enableInterrupts();  // Re-enable interrupts
}
```

- `disableInterrupts()`: Prevents context switches and interrupt handling while the critical section is in progress.
- `enableInterrupts()`: Restores normal interrupt handling after the critical section.

This approach works well in uniprocessor systems where there is no need for complex synchronization primitives, but can be inefficient or unsafe on multiprocessor systems.

Take 3: Software Implementation with Hardware Support

This approach assumes the presence of hardware support for atomic operations such as **test-and-set** or **compare-and-swap**. These atomic operations allow a processor to atomically read a memory value, modify it, and write the new value, all without the risk of interruptions from other processes or threads.

Atomic Lock Acquisition with Hardware Support

```
void lock_acquire(lock_t *L) {
    while (true) {
        if (atomic_test_and_set(&L->value, BUSY) == FREE) {
            // Successfully acquired the lock
            break;
        }
        L->waiting.add(myTCB);  // Add the current thread to
the waiting list
```

```
        suspend();  // Suspend the current thread
    }
}
```

Lock Release with Hardware Support

```
void lock_release(lock_t *L) {
    if (!L->waiting.empty()) {
        thTCB = L->waiting.remove();  // Get the next thread
from the waiting list
        readyList.append(thTCB);  // Add the thread to the re
ady list
    } else {
        atomic_clear(&L->value);  // Atomically release the l
ock (set to FREE)
    }
}
```

- `atomic_test_and_set(&L->value, BUSY)`: Atomically tests the value of `L->value` and sets it to `BUSY` if it's currently `FREE`. If it was already `BUSY`, the thread will keep spinning (busy-waiting) or suspend itself.
- `atomic_clear(&L->value)`: Atomically sets `L->value` back to `FREE` to release the lock.

Hardware support like this allows the synchronization primitives to be implemented efficiently, with minimal overhead and no need to disable interrupts.

Lock Structure (`lock_t`)

The `lock_t` structure used in the examples can be defined as follows:

```
typedef struct lock {
    int value;  // The state of the lock (BUSY or FREE)
    Queue waiting;  // A queue of threads waiting for the loc
k
} lock_t;
```

```
#define FREE 0
#define BUSY 1
```

In this structure:

- `value` indicates whether the lock is free (`FREE`) or busy (`BUSY`).
- `waiting` is a queue of threads that are waiting for the lock when it is busy.

Multiprocessor Synchronization

In a multiprocessor system, **disabling interrupts** is not sufficient to ensure mutual exclusion because multiple processors can execute code simultaneously, making it possible for them to interfere with each other even if interrupts are disabled.

To solve this, we need **atomic operations** provided by **Read-Modify-Write (RMW) instructions**. These instructions ensure that a value in memory is read, modified, and written back to memory without being interrupted by another processor.

Read-Modify-Write (RMW) Instructions

RMW operations are essential in multiprocessor systems for ensuring proper synchronization. These operations allow a processor to:

- **Read** a value from memory,
- **Modify** that value, and
- **Write** the modified value back to memory, all in a single, atomic step.

By using these operations, we can ensure that no other processor can intervene and modify the memory value while this sequence is in progress. This is critical for maintaining data integrity in concurrent environments.

Examples of RMW Instructions:

1. **Test-and-Set (TS)**: This is one of the most common atomic operations. It reads a value, sets it to a new value (typically `1`), and then writes it back to memory. It's commonly used in lock implementations.

- ***Intel***: `xchgb` (Exchange Byte) with the `lock` prefix for atomicity.
- ***ARM***: `LDREX`/`STREX` (Load-Exclusive/Store-Exclusive) pair of instructions, used for building atomic operations like compare-and-swap.

2. **Compare-and-Swap (CAS)**: This operation reads a value from memory, compares it to a specified value, and if they are equal, it swaps the value in memory with a new value. This is widely used for lock-free data structures.
   - ***Intel***: `cmpxchg` (Compare and Exchange).
   - ***ARM***: `LDREX`/`STREX` operations can also be used for implementing compare-and-swap.

Do the Types of RMW Instructions Matter?

For the **implementation of locks** and **condition variables**, the specific type of RMW instruction used does not typically matter. Any atomic read-modify-write operation that guarantees atomicity between the read and write phases can be used to implement synchronization primitives.

Both **Test-and-Set** and **Compare-and-Swap** can be used for constructing mutual exclusion mechanisms (like spinlocks) and condition variable operations, depending on the architecture and the required level of complexity.

However, the difference between the types of RMW instructions can have an impact on the efficiency and flexibility of certain synchronization patterns:

- **Test-and-Set (TS)** is simple and often used in simple locking mechanisms, like spinlocks.
- **Compare-and-Swap (CAS)** is more flexible and can be used for building more complex lock-free and wait-free data structures.

Example: Test-and-Set (TS) for Locks

Here's a simple example of how a **Test-and-Set** operation could be used to implement a lock on a multiprocessor system:

```
// Lock structure with a Test-and-Set operation
typedef struct lock {
    volatile int flag;  // A flag to indicate whether the loc
```

```
k is acquired
} lock_t;

void lock_acquire(lock_t *L) {
    while (test_and_set(&L->flag)) {
        // Busy-waiting until the lock is free
    }
}

void lock_release(lock_t *L) {
    L->flag = 0;  // Release the lock by resetting the flag
}
```

In this example:

- `test_and_set(&L->flag)` atomically sets `L->flag` to `1` and returns its previous value. If the previous value is `1`, the lock is already held, and the thread will keep spinning until it can acquire the lock.
- `L->flag = 0` is used to release the lock.

This simple lock works because the `test_and_set` operation ensures that only one processor can successfully set the flag at a time, preventing race conditions.

Hardware Support for RMW Operations

In **multiprocessor systems**, **RMW instructions** are directly supported by hardware, which makes these operations fast and efficient for managing synchronization.

- **Intel (x86/x64 architectures)**: The `xchg` (exchange) instruction with the `lock` prefix allows atomic read-modify-write operations. The `lock` prefix ensures that the operation is atomic and prevents other processors from accessing the memory location simultaneously.
- **ARM**: The `LDREX` (Load-Exclusive) and `STREX` (Store-Exclusive) instructions are used in a similar manner to test-and-set or compare-and-swap. These operations allow a processor to perform an atomic read-modify-write cycle and can be used to implement synchronization primitives.

Spinlocks are a simple locking mechanism used primarily in operating systems and low-level concurrent programming. Unlike regular locks that might cause a thread to sleep (and thus involve context switches), a spinlock makes the processor wait in a loop (i.e., "spin") until the lock becomes free. This is known as **active waiting**.

Characteristics of Spinlocks

- **Active Waiting:**

  The processor continuously checks (spins) in a loop for the lock to become free. This approach is effective if the lock is held for a very short time.

- **Low Overhead:**

  Spinlocks are useful in scenarios where the overhead of context switching is greater than the cost of simply waiting in a loop.

- **Use Cases:**

  Spinlocks are often used in the OS kernel—for example, to protect data structures like the ready list or to implement other synchronization primitives.

Simple Implementation of Spinlocks

Below is a simple example of how you might implement a spinlock in C/C++ using a Test-And-Set (TAS) operation:

```
// Definitions for lock states
#define FREE 0
#define BUSY 1

// A spinlock type
typedef volatile int spinlock_t;

// TestAndSet function simulates an atomic operation that set
s the lock
// and returns the previous value.
```

```
int TestAndSet(volatile int *lock) {
    int old = *lock;
    *lock = BUSY;
    return old;
}

// Acquire the spinlock by continuously trying until it's fre
e
void SpinlockAcquire(spinlock_t *SL) {
    // Spin (active waiting) until TestAndSet returns FREE, m
eaning the lock was
      // not held
    while (TestAndSet(SL) == BUSY)
        ;  // Busy-wait (spin)
}

// Release the spinlock and use a memory barrier to ensure me
mory ordering
void SpinlockRelease(spinlock_t *SL) {
    *SL = FREE;    // Mark the lock as free
    // Memory barrier ensures that all memory operations comp
lete before the lock
    // is released.
    memory_barrier();
}
```

Explanation of the Code:

- **TestAndSet:**

  This function performs an atomic operation that sets the lock value to `BUSY` while returning its previous value. If the previous value is `BUSY`, it means the lock was already held.

- **SpinlockAcquire:**

  The function enters a `while` loop, continuously calling `TestAndSet` until it finds the lock in the `FREE` state. Once `TestAndSet` returns `FREE`, the lock is acquired.

- **SpinlockRelease:**

  When releasing the lock, the function sets the spinlock variable back to `FREE`. A memory barrier (which is often implemented as a hardware-specific instruction) is used to ensure that all memory operations are completed before releasing the lock. This prevents reordering of memory operations that could otherwise lead to subtle bugs.

When to Use Spinlocks

- **Short Critical Sections:**

  They work best when the lock is held for a very short duration. If the critical section is long, the CPU might waste cycles spinning.

- **Low Contention:**

  In scenarios where locks are not heavily contended, spinlocks can be very efficient.

- **Kernel or Interrupt Context:**

  Spinlocks are commonly used in kernel code where sleeping is not an option, such as in interrupt handlers or when protecting data structures that are accessed in high-priority contexts.

Key Considerations

- **Active Waiting:**

  Since spinlocks use active waiting, they can lead to high CPU usage if many threads are spinning for a long time. They are not ideal for situations where locks might be held for extended periods.

- **Multiprocessor Systems:**

  Spinlocks are useful in multiprocessor environments, but care must be taken to ensure that the busy-wait loop does not monopolize CPU resources.

- **Memory Barriers:**

  Ensuring proper memory ordering is crucial in multiprocessor systems. The use of a memory barrier in `SpinlockRelease` is essential to prevent reordering of

memory operations, ensuring that all writes are visible before the lock is released.

Spinlocks are a simple locking mechanism used primarily in operating systems and low-level concurrent programming. Unlike regular locks that might cause a thread to sleep (and thus involve context switches), a spinlock makes the processor wait in a loop (i.e., "spin") until the lock becomes free. This is known as **active waiting**.

Characteristics of Spinlocks

- **Active Waiting**:

  The processor continuously checks (spins) in a loop for the lock to become free. This approach is effective if the lock is held for a very short time.

- **Low Overhead**:

  Spinlocks are useful in scenarios where the overhead of context switching is greater than the cost of simply waiting in a loop.

- **Use Cases**:

  Spinlocks are often used in the OS kernel—for example, to protect data structures like the ready list or to implement other synchronization primitives.

Simple Implementation of Spinlocks

Below is a simple example of how you might implement a spinlock in C/C++ using a Test-And-Set (TAS) operation:

```
// Definitions for lock states
#define FREE 0
#define BUSY 1

// A spinlock type
typedef volatile int spinlock_t;

// TestAndSet function simulates an atomic operation that set
```

```
s the lock
// and returns the previous value.
int TestAndSet(volatile int *lock) {
    int old = *lock;
    *lock = BUSY;
    return old;
}

// Acquire the spinlock by continuously trying until it's fre
e
void SpinlockAcquire(spinlock_t *SL) {
    // Spin (active waiting) until TestAndSet returns FREE, m
eaning the lock was
      // not held
    while (TestAndSet(SL) == BUSY)
        ;  // Busy-wait (spin)
}

// Release the spinlock and use a memory barrier to ensure me
mory ordering
void SpinlockRelease(spinlock_t *SL) {
    *SL = FREE;    // Mark the lock as free
    // Memory barrier ensures that all memory operations comp
lete before the lock
        // is released.
    memory_barrier();
}
```

Explanation of the Code:

- **TestAndSet**:

  This function performs an atomic operation that sets the lock value to `BUSY` while returning its previous value. If the previous value is `BUSY`, it means the lock was already held.

- **SpinlockAcquire**:

The function enters a `while` loop, continuously calling `TestAndSet` until it finds the lock in the `FREE` state. Once `TestAndSet` returns `FREE`, the lock is acquired.

- **SpinlockRelease**:

  When releasing the lock, the function sets the spinlock variable back to `FREE`. A memory barrier (which is often implemented as a hardware-specific instruction) is used to ensure that all memory operations are completed before releasing the lock. This prevents reordering of memory operations that could otherwise lead to subtle bugs.

When to Use Spinlocks

- **Short Critical Sections**:

  They work best when the lock is held for a very short duration. If the critical section is long, the CPU might waste cycles spinning.

- **Low Contention**:

  In scenarios where locks are not heavily contended, spinlocks can be very efficient.

- **Kernel or Interrupt Context**:

  Spinlocks are commonly used in kernel code where sleeping is not an option, such as in interrupt handlers or when protecting data structures that are accessed in high-priority contexts.

Key Considerations

- **Active Waiting**:

  Since spinlocks use active waiting, they can lead to high CPU usage if many threads are spinning for a long time. They are not ideal for situations where locks might be held for extended periods.

- **Multiprocessor Systems**:

  Spinlocks are useful in multiprocessor environments, but care must be taken to ensure that the busy-wait loop does not monopolize CPU resources.

- **Memory Barriers**:

> Ensuring proper memory ordering is crucial in multiprocessor systems. The use of a memory barrier in `SpinlockRelease` is essential to prevent reordering of memory operations, ensuring that all writes are visible before the lock is released.

Test-and-Set Lock (TSL) Example

Assume the processor provides an atomic TSL instruction with the following behavior:

```
TSL REG, &ADDR   // REG ← MEMORY[ADDR]
                 // MEMORY[ADDR] ← BUSY
```

This single instruction does two things:

1. Reads the current value at the address (into a register).
2. Atomically writes the value BUSY (1) into that memory location.

Spinlock Implementation with TSL

In our spinlock, the memory cell referenced by `spinLockValue` holds either FREE (0) or BUSY (1). The implementation using TSL might look like this:

```
// spinLockValue is a memory cell containing a binary value:
FREE (0) or BUSY (1)
// The TSL instruction writes the current value at &spinLockV
alue into register R
// and writes BUSY (1) into &spinLockValue atomically.

void spinlockAcquire(volatile int *spinLockValue) {
Loop:
    TSL R, spinLockValue;       // Atomically: R = *spinLockV
alue, and *spinLock

// alue becomes BUSY
```

```
    CMP R, #BUSY;                // Compare the original value
with BUSY (1)
    BEQ Loop;                    // If R equals BUSY, branch b
ack to Loop

// (spin until free)
    // When the loop exits, it means TSL returned
    // FREE and the lock is now acquired.
    END;                        // (Pseudo-instruction: lock i
s acquired)
}

void spinlockRelease(volatile int *spinLockValue) {
    MOV #FREE, spinLockValue;   // Write FREE (0) to unlock
    MFENCE;                     // Memory barrier to ensure a
ll

// writes complete before releasing
}
```

Explanation:

- `spinlockAcquire()`:
  - The code continuously loops (spins) while executing the TSL instruction.
  - TSL atomically sets the lock value to BUSY and returns its previous state.
  - If the previous state was BUSY, the thread continues to spin.
  - When TSL returns FREE, the thread has successfully acquired the lock.
- `spinlockRelease()`:
  - The lock is released by simply writing FREE (0) to the memory cell.
  - A memory barrier (`MFENCE`) is used to ensure that all memory operations are completed before the lock is actually released.

Multiprocessor Lock Implementation Using Spinlock

In a multiprocessor system, simply disabling interrupts is not enough. Instead, the lock implementation uses a spinlock (implemented with TSL) to protect the higher-level lock structure.

Consider a `lock_t` structure that contains:

- A `value` field that indicates if the lock is BUSY or FREE.
- A waiting queue (e.g., a list of thread control blocks, TCBs) for threads that cannot acquire the lock.

Here's an outline of how the lock acquire and release functions might work:

`LockAcquire()`

```
LockAcquire(lock_t *L) {
    disableInterrupts();                    // Prevent context
switching
    spinlockAcquire(&L->spinLock);            // Acquire the s
pinlock for the lock

// structure

    if (L->value == BUSY) {                   // Check if the
high-level lock is

// lready held

        L->waiting.add(myTCB);                // Add current t
hread (myTCB) to the

// waiting queue

        sched_suspend(&L->spinLock);          // Suspend curre
nt thread (spinlock

// remains held during scheduling)
    } else {
```

```
        L->value = BUSY;                        // Mark the lock
as BUSY
        spinlockRelease(&L->spinLock);          // Release the
spinlock for the lo

// ck structure
    }
    enableInterrupts();                     // Re-enable inter
rupts
}
```

LockRelease()

```
LockRelease(lock_t *L) {
    disableInterrupts();                    // Prevent context
switching
    spinlockAcquire(&L->spinLock);          // Acquire the s
pinlock to modify

// the lock structure

    if (!L->waiting.empty()) {              // Check if there
are waiting threads
        thTCB = L->waiting.remove();        // Remove the fi
rst waiting thread

// from the queue

        sched_resume(thTCB);                // Resume the wa
iting thread

// (scheduler will mark it as ready)
    } else {
        L->value = FREE;                    // No threads wa
iting, mark the lock
```

```
// as FREE
    }
    spinlockRelease(&L->spinLock);           // Release the s
pinlock for the lock

//  structure

    enableInterrupts();                     // Re-enable inter
rupts
}
```

Key Points:

- **Disabling Interrupts:**

  This prevents the current thread from being preempted while it is acquiring or releasing the lock. However, in a multiprocessor system, this is not sufficient alone, hence the use of the spinlock.

- **Spinlock Protection:**

  The spinlock (`L->spinLock`) protects the shared fields (`L->value` and `L->waiting`) of the lock structure. The higher-level lock logic is thus shielded from concurrent access across processors.

- **Waiting Queue and Scheduler:**

  If the lock is BUSY, the current thread is added to a waiting queue and suspended. When the lock is released, the scheduler resumes one of the waiting threads (according to the policy, e.g., FIFO).

- **Memory Barriers and Atomicity:**

  The use of RMW instructions and memory barriers ensures that the operations are performed atomically and that the memory state is consistent across processors.

In Linux, lock implementations are often designed with two distinct paths: a **fast path** for the common case where the lock is available, and a **slow path** for when the lock is contended or when additional coordination is needed.

Fast Path

- **When It's Used:**

  The fast path is taken when the lock is free and no threads are waiting.

- **Mechanism:**

  A thread uses a Test-And-Set (or another Read-Modify-Write operation) to quickly acquire the lock without invoking heavier mechanisms.

- **Example Pseudocode:**

```
// Fast path for acquiring the lock
if (lock == FREE && no_waiters()) {
    if (TestAndSet(&lock) == FREE) {
        // Lock acquired successfully
        return;
    }
}
// Otherwise, fall back to the slow path
```

This fast path minimizes overhead because it avoids system calls and kernel involvement when there is no contention.

Slow Path

- **When It's Used:**
  The slow path is invoked when the lock is already BUSY or if there are threads waiting for the lock.
- **Mechanism:**
  On the slow path, the thread may:
  - Suspend itself.
  - Be added to a waiting queue.

- Potentially trigger a system call to let the kernel manage blocking and wake-ups.

- **Reference:**
  The slow path logic builds on the concepts we saw in previous slides (or explanations) regarding waiting queues and scheduler interactions.

User-Level Locks

- **User-Level Fast Path:**
  A user-level lock typically attempts to acquire the lock using a fast, atomic operation such as Test-And-Set.
- **User-Level Slow Path:**
  If the fast path fails (because the lock is BUSY or there are waiters), the thread makes a system call to the kernel. The kernel then handles the lock acquisition via a kernel lock, which may involve blocking the thread and performing context switches.

Summary

- **Fast Path (Common Case):**
  - Check if the lock is FREE and no one is waiting.
  - Use an atomic operation like Test-And-Set to acquire the lock.
  - Avoid kernel involvement to keep the operation fast and lightweight.
- **Slow Path (Contended Case):**
  - If the lock is BUSY or there are waiters, handle the situation with more robust mechanisms.
  - This often involves system calls and kernel-level locking to manage thread scheduling and wake-ups.
- **User-Level Locks:**
  - Try the fast path with atomic operations.
  - Fall back to a kernel-assisted slow path via a system call when necessary.

This design allows the Linux locking mechanisms to perform very efficiently in the uncontended case while still providing robust synchronization support when locks become highly contended

Semaphores

A **semaphore** is a synchronization primitive that maintains a non-negative integer value along with a waiting queue. The two primary operations on a semaphore are:

- `P()` **(or wait/decrement):**

  Atomically waits until the semaphore's value is greater than 0, then decrements it. If the value is 0, the caller blocks (or is suspended) until the semaphore is available.

- `V()` **(or signal/increment):**

  Atomically increments the semaphore's value. If there are threads waiting, one is awakened (i.e., removed from the waiting queue) instead of simply incrementing the value.

Data Structures:

A semaphore consists of:

- An **integer value** that represents the available count.
- A **queue** to hold threads (or processes) waiting on the semaphore.

Example Behavior:

If a semaphore is initialized to 1 (a binary semaphore), then two consecutive calls to P() will result in the semaphore value becoming 0, and the second call will block if another thread tries to decrement it.

Semaphores are especially useful in scenarios like:

- **Unlocked wait:** For instance, in interrupt handlers.
- **Fork/join:** To coordinate the completion of concurrent tasks.

P and V Implementation for Multiprocessor Systems

In a multiprocessor environment, we cannot simply disable interrupts because other processors may still access shared memory concurrently.

Therefore, we use atomic Read-Modify-Write (RMW) operations—often protected with a spinlock—and disable interrupts locally when needed. Below is a pseudocode implementation for P and V using a spinlock to protect the semaphore structure.

P (Wait) Operation:

```
P(sem) {
    disableInterrupts();                // Prevent context sw
itching locally
    spinLockAcquire(&spinLock);         // Acquire spinlock t
o

// protect semaphore data

    if (sem->value == 0) {              // If semaphore is ze
ro, nothing

// available

        sem->waiting.add(myTCB);        // Add current threa
d's control

// block to waiting queue

        suspend(&spinLock);             // Suspend the threa
d; this call

// releases spinlock as part of

// context switch
    } else {
```

```
        sem->value--;                    // Decrement semaphor
e value

// (resource is acquired)

        spinLockRelease(&spinLock);      // Release spinlock
    }

    enableInterrupts();                  // Re-enable interrup
ts
}
```

V (Signal) Operation:

```
V(sem) {
    disableInterrupts();                 // Prevent interrupti
ons during

// critical section

    spinLockAcquire(&spinLock);          // Acquire spinlock t
o protect

// semaphore data

    if (!sem->waiting.empty()) {         // If there are threa
ds waiting
        thTCB = sem->waiting.remove();   // Remove one thread
from the

// waiting queue

        readyList.append(thTCB);         // Mark that thread a
s ready to run
    } else {
```

```
        sem->value++;                    // Otherwise, increme
nt the

// semaphore value
    }

    spinLockRelease(&spinLock);          // Release spinlock
    enableInterrupts();                  // Re-enable interrup
ts
}
```

Notes:

- The implementation invokes the scheduler and performs a context switch when suspending a thread.
- It “throws” the semaphore to the awakened thread by ensuring it is now in the ready list.

Semaphore-Based Bounded Buffer Example

The classic **Producer-Consumer** problem (or bounded buffer problem) can be implemented with semaphores. Here’s one common approach:

Shared Variables:

- `buf[]`: The buffer array.
- `front`, `last`: Indexes into the buffer.
- `size`: The capacity of the buffer.
- `empty`: A semaphore initialized to the number of buffer slots (i.e., available empty slots).
- `full`: A semaphore initialized to 0 (i.e., no full slots).
- `mutex`: A semaphore initialized to 1 for mutual exclusion on the buffer.

Consumer (`get()`) Operation:

```
get() {
    empty.P();                  // Wait until there is an it
em

// (empty semaphore is decremented)

    mutex.P();                  // Enter critical section fo
r accessing

// the buffer

    item = buf[front];          // Get the item from the buf
fer
    front = (front + 1) % size; // Move front index (circula
r buffer)
    mutex.V();                  // Leave critical section
    full.V();                   // Signal that there is one
more empty

// slot (increment full semaphore)
    return item;
}
```

Producer (`put(item)`) Operation:

```
put(item) {
    full.P();                  // Wait until there is an emp
ty slot
                                                        // (f
ull semaphore)
    mutex.P();                 // Enter critical section for
modifying

// the buffer
```

```
    buf[last] = item;           // Put the item in the buffer
    last = (last + 1) % size;   // Move last index (circular
buffer)
    mutex.V();                  // Leave critical section

    empty.V();                  // Signal that there is one m
ore

// item available (increment empty semaphore)
}
```

Initialization:

- `front = last = 0`
- `empty` is initialized to the buffer capacity.
- `full` is initialized to 0.
- `mutex` is initialized to 1.

Implementing Condition Variables Using Semaphores

The goal of condition variables is to allow threads to wait for some condition on shared data while releasing the associated lock and to be awakened when the condition is signaled. Below are several attempts at using semaphores to implement condition variables, with a discussion of their correctness.

Take 1:

```
wait(lock) {
    lock.release();  // Release the lock
    sem.P();         // Wait on the semaphore
    lock.acquire();  // Re-acquire the lock after being signa
led
}
```

```
signal() {
    sem.V();          // Signal the semaphore
}
```

Problem:

This implementation is **not correct** because condition variables are supposed to be memoryless. If a thread signals when no one is waiting, nothing should happen; but in this case, the semaphore would be incremented. Later, a waiter might call P() and immediately proceed without the condition being true.

Take 2:

```
wait(lock) {
    lock.release();  // Release the lock
    sem.P();         // Wait on the semaphore
    lock.acquire();  // Re-acquire the lock
}

signal() {
    if (semaphore is not empty)  // Check if any thread is wa
iting
        sem.V();                 // Signal the semaphore
}
```

Problem:

This version is also **not correct**. A race condition exists because the signaler could slip in between the time the waiting thread releases the lock and actually executes the P() call, causing the semaphore to appear empty. This means the signal may be lost and the waiter might never be woken up.

Take 3: Correct Implementation

A correct implementation requires each waiting thread to create its own semaphore and then be placed in a waiting queue. When a thread signals, it picks one semaphore from the queue and signals that specific semaphore.

```
wait(lock) {
    selfSem = createSemaphore();   // Create a new semaphore
for this waiter
    queue.append(selfSem);         // Append it to the condit
ion variable's

// waiting queue

    lock.release();                // Release the associated
lock
    selfSem.P();                   // Wait on this semaphore
    destroySemaphore(selfSem);     // Clean up after being wo
ken up
    lock.acquire();                // Re-acquire the lock bef
ore returning
}

signal() {
    if (!queue.empty()) {
        selfSem = queue.remove();  // Remove the first semaph
ore in the

// waiting queue

        selfSem.V();               // Signal that semaphore t
o wake up

// the waiting thread
    }
}
```

Key Points for the Correct Implementation:

- Each waiting thread creates its own semaphore (`selfSem`).

- The waiting thread enqueues its semaphore in the condition variable's waiting queue.
- The thread releases the lock and then waits on its own semaphore.
- The signal operation checks if the waiting queue is not empty and then signals the first waiting thread by performing V() on its semaphore.
- After being awakened, the waiting thread destroys its semaphore and re-acquires the lock.

This approach prevents race conditions and ensures that the condition variable remains memoryless—signals are only delivered to actual waiters.